DOCKER PRACTICAL & TROUBLESHOOTING TEST

30 Scenario-Based Single-Choice Questions with Detailed Answer Key

Based on Official Docker Documentation

October 11, 2026

Test Overview:

- **Format:** 30 Single-Choice Questions (choose the single best answer: A, B, C, or D).
- **Focus:** Practical CLI usage, error traces, common misconceptions, storage/networking internals, Dockerfile caching, multi-stage builds, and multi-container operations.
- **Answer Key:** Full answers and technical explanations are provided in the final section of this PDF.

Part 1: Basic Concepts & Container Lifecycle (Questions 1–7)

Question 1. You start an interactive container using `docker run -i -t ubuntu /bin/bash`, create a file at `/tmp/debug.log`, and then type `exit` in the terminal. When you run `docker ps`, the container does not appear in the list. What is the current state of the container and `/tmp/debug.log`?

A. The container is automatically deleted upon exiting `/bin/bash`, and `/tmp/debug.log` is permanently lost.
B. The container is stopped (`Exited`), and `/tmp/debug.log` remains intact in its writable layer until the container is explicitly removed.
C. The container continues running in the background, and `/tmp/debug.log` is moved to the default Docker volume.
D. The container is paused in host RAM, and `/tmp/debug.log` is committed as a new layer in the `ubuntu` image.

Question 2. A developer runs an Alpine Linux container on an Ubuntu Linux host machine that uses kernel version `6.8.0-45-generic`. Inside the Alpine container, the developer executes `uname -r`. What output will they see, and why?

A. `alpine-virt-3.20`, because each container boots its own lightweight Alpine guest kernel.
B. `6.8.0-45-generic`, because containers are isolated processes that share the host operating system's kernel.
C. An error message, because `uname` requires hypervisor hardware virtualization privileges.
D. `dockerd-virtual-kernel`, because the Docker daemon emulates a generic kernel for all containers.

Question 3. While attempting to list running containers with `docker ps` on a Linux server, you receive the following error trace:

```
Cannot connect to the Docker daemon at unix:///var/run/docker.sock.
Is the docker daemon running?
```

Which statement accurately describes why this error occurs based on Docker's architecture?

- **A.** The Docker CLI client (`docker`) works standalone, but the local Docker Hub registry cache is corrupted.
- **B.** The Docker client (`docker`) could not communicate via REST API over the UNIX socket because the background server process (`dockerd`) is stopped or inaccessible.
- **C.** The container's internal network namespace has blocked port 2375 on the loopback interface.
- **D.** Docker Compose has locked the socket file while reconciling a `compose.yaml` file.

Question 4. On Docker Hub, a repository page indicates that an image tag has a size of 112 MB. However, after running `docker pull` and inspecting the local image with `docker image ls`, the SIZE column reports 318 MB. Why is there a discrepancy?

- **A.** `docker image ls` pre-allocates a 200 MB writable container layer for every pulled image.
- **B.** Docker Hub displays the compressed download size of the layers, whereas `docker image ls` displays the uncompressed size of the image extracted on disk.
- **C.** `docker pull` automatically downloads all historical tags in the repository alongside the requested tag.
- **D.** The local image includes the size of the Docker Engine runtime binaries injected during extraction.

Question 5. You are testing a multi-container application with a MySQL database defined in `compose.yaml` using a named volume `mysql_data`. After inserting broken test records, you run `docker compose down` and then `docker compose up -d`, expecting a clean database. However, the broken test records are still there. Why?

- **A.** `docker compose down` only stops containers without removing them; you must run `docker compose stop --clean`.
- **B.** By default, `docker compose down` removes containers and networks but preserves named volumes to prevent accidental data loss; you must pass `--volumes` to delete the volume.
- **C.** MySQL automatically commits its database tables into the base Docker image before shutting down.
- **D.** Docker Compose caches database rows inside the `compose.yaml` metadata file.

Question 6. A developer logs into Docker Hub via `docker login` under the username `alexdev`. They build an image using `docker build -t payment-service:1.0 .` and immediately run `docker push payment-service:1.0`. The command fails with:

```
denied: requested access to the resource is denied
```

What is the root cause of this failure?

- **A.** The tag `1.0` is invalid; Docker Hub only accepts `latest` on the first push.
- **B.** Because no namespace was specified in the tag, Docker resolved the full image path to `docker.io/library/payment-service:1.0`, and `alexdev` does not have permission to push to the official `library` namespace.
- **C.** `docker push` requires the `--registry docker.io` flag when pushing custom images.
- **D.** The developer did not run `docker container commit` before pushing the image.

Question 7. Which statement best distinguishes a `Dockerfile` from a `compose.yaml` file?

A. A `Dockerfile` is used only in development, whereas `compose.yaml` is used exclusively for building base OS images.
B. A `Dockerfile` provides instructions to build a read-only container image, while a `compose.yaml` file declaratively defines how to run and connect one or more containers, networks, and volumes.
C. A `Dockerfile` manages virtual machines, whereas `compose.yaml` manages Linux kernel namespaces.
D. A `compose.yaml` file replaces the need for container images by executing source code directly on the host OS.

Part 2: Docker Engine — Storage & Backends (Questions 8–15)

Question 8. The official `nginx` image contains a default `/usr/share/nginx/html/index.html` file. Compare two commands run on a host where `/tmp/empty_dir` is a newly created empty directory and `empty_vol` is a newly created empty Docker volume:

- **Command 1:** `docker run -d -p 8081:80 -v empty_vol:/usr/share/nginx/html nginx`
- **Command 2:** `docker run -d -p 8082:80 -v /tmp/empty_dir:/usr/share/nginx/html nginx`

When you access `http://localhost:8081`, you see the default Nginx welcome page. When you access `http://localhost:8082`, you receive `403 Forbidden`. Why?

A. Bind mounts automatically block port 80 unless the `:rw` flag is explicitly appended.
B. Mounting an empty volume into a non-empty container directory propagates the existing container files into the volume, whereas a bind mount obscures (hides) the existing files with the empty host directory.
C. Docker volumes automatically download a fresh `index.html` from Docker Hub during startup.
D. `/tmp/empty_dir` is converted into a `tmpfs` mount that deletes `index.html`.

Question 9. The directory `/opt/webapp/config` does **not** currently exist on your Linux host. You execute the following command:

```
docker run -d --mount type=bind,source=/opt/webapp/config,target=/app/config nginx
```

What happens when this command is executed, and how does it differ from using `-v /opt/webapp/confi`

A. Both `--mount` and `-v` fail with an error because Docker never creates host directories.
B. `--mount` fails with an error stating that the bind source path does not exist, whereas `-v` automatically creates the missing `/opt/webapp/config` directory on the host.
C. `--mount` automatically creates `/opt/webapp/config` on the host, whereas `-v` converts it into a named volume.
D. `--mount` falls back to a `tmpfs` mount in RAM if the host directory is missing.

Question 10. A container running under the default `overlay2` storage driver needs to modify a single line inside a `500 MB` file that resides in a lower read-only image layer (without any volume or bind mount attached). What happens at the storage driver level the first time the container writes to this file?

A. The storage driver modifies the `500 MB` file directly inside the lower read-only layer, affecting all other containers sharing that image.
B. The storage driver performs a Copy-on-Write (`copy_up`) operation, copying the entire `500 MB` file from the lower read-only layer into the container's writable layer before applying the modification, which can introduce noticeable I/O latency.
C. Only the modified 4 KB disk block is written to RAM, leaving the writable container layer at 0 bytes.
D. The write operation fails with `Read-only file system` unless a volume is attached.

Question 11. You want a Linux container to process sensitive API tokens in a temporary directory `/run/tokens` such that the files are never written to the host disk or the container's writable filesystem, and are immediately wiped if the container stops. You also want to restrict this directory to a maximum size of `64 MB`. Which option should you use?

A. `--tmpfs /run/tokens:size=64m` (using the standalone `--tmpfs` flag with options)
B. `--mount type=tmpfs,destination=/run/tokens,tmpfs-size=67108864`
C. `--mount type=volume,dst=/run/tokens,readonly`
D. `--mount type=image,src=tokens:latest,dst=/run/tokens`

Question 12. You need to debug a minimal distroless production container that has no shell or debugging utilities installed. You decide to attach the `/bin` folder of the `busybox:latest` image into `/debug-tools` using an image mount:

```
docker run --rm -it --mount type=image,src=busybox:latest,dst=/debug-
tools,image-subpath=bin my-minimal-app /debug-tools/sh
```

Which two prerequisites **must** be met for this command to succeed?

A. The `vfs` storage driver must be enabled, and the mount must be configured as read-write (`rw`).
B. The `containerd` image store must be enabled on the daemon, and `busybox:latest` must already be pulled into the local image store beforehand (Docker does not auto-pull missing images for image mounts).
C. Docker Swarm mode must be active, and `my-minimal-app` must run with `--network host`.
D. You must use the `-v` shorthand flag instead of `--mount`, because `--mount` does not support `image-subpath`.

Question 13. A system administrator enables the `containerd` image store on an existing Linux server by adding `"features": {"containerd-snapshotter": true}` to `/etc/docker/daemon.json` and restarting the Docker service. Immediately afterward, running `docker images` and `docker ps -a` returns empty lists, yet the disk space on `/var/lib/docker` is still consumed. Why?

A. Enabling `containerd-snapshotter` formats the host partition and corrupts existing ext4 inodes.
B. `containerd` uses a separate storage directory for snapshotters; images and containers created previously under the classic storage driver (`overlay2`) remain on disk in the old driver directory but are hidden while the `containerd` store is active.
C. The Docker CLI must be downgraded to version 19.03 to read `containerd` snapshotter metadata.
D. All existing images were automatically pushed to Docker Hub and locked remotely.

Question 14. Why does pulling an image when using the `containerd` image store typically

consume more local disk space than when using the classic `overlay2` graph driver?

- **A.** Because `containerd` disables Copy-on-Write and duplicates full directories like the `vfs` driver.
- **B.** Because `containerd` retains both the compressed image layers downloaded from the registry and the unpacked (extracted) layers on disk.
- **C.** Because `containerd` automatically downloads every architecture variant (`amd64`, `arm64`, `s390x`) for every pulled image.
- **D.** Because `containerd` creates a 1 GB swap file for every layer in the image.

Question 15. You run `docker ps` and confirm that zero containers are currently running. However, when you try to delete a volume using `docker volume rm postgres_data`, Docker outputs:

```
Error response from daemon: remove postgres_data: volume is in use
- [7a070c9ffeaa]
```

What is causing this error, and how should you resolve it?

- **A.** A stopped container (`7a070c9ffeaa`) is still referencing the volume; you must remove the stopped container using `docker rm 7a070c9ffeaa` before removing the volume.
- **B.** The volume is locked by Docker Hub; you must run `docker logout` first.
- **C.** You must delete the volume's files directly from `/var/lib/docker/volumes/postgres_data` on the host OS using `rm -rf`.
- **D.** `docker volume rm` only works on anonymous volumes, not named volumes.

Part 3: Docker Engine — Networking & DNS (Questions 16–20)

Question 16. A developer starts two containers on the same host without specifying any `--network` flags:

```
docker run -d --name redis-cache redis
docker run -d --name node-api -p 3000:3000 my-node-api
```

Inside `node-api`, the application attempts to connect to `redis://redis-cache:6379`, but crashes with `Error: getaddrinfo ENOTFOUND redis-cache`. Why did hostname resolution fail?

- **A.** Containers cannot communicate with each other on the same host unless port 6379 is published to the host with `-p 6379:6379`.
- **B.** Both containers attached to the default `bridge` network, which does not provide automatic DNS resolution by container name; they must be attached to a custom user-defined network to resolve `redis-cache` via Docker's embedded DNS.
- **C.** The `redis` container must be started with the `--expose-dns` flag.
- **D.** Container names containing hyphens (`-`) are invalid DNS hostnames in Docker.

Question 17. You attach a container to a user-defined bridge network (`docker network create app-net`) and inspect `/etc/resolv.conf` inside the container. It contains `nameserver 127.0.0.11`. How does this DNS configuration work?

- **A.** `127.0.0.11` is Docker's embedded DNS server, which resolves names and aliases of other

containers on app-net and forwards external DNS lookups to the DNS servers configured on the host machine.

B. 127.0.0.11 blocks all external internet access and only allows loopback connections inside the same container.

C. 127.0.0.11 is only used for IPv4 networks; IPv6-only custom networks do not use the embedded DNS server.

D. 127.0.0.11 is a static copy of the host's /etc/hosts file that never updates when new containers join the network.

Question 18. You want to run a network debugging sidecar container that shares the exact same network stack (IP address and port space) as an already running container named web-server. You run:

```
docker run -it --network container:web-server --hostname debug-sidecar
nicolaka/netshoot
```

The command fails immediately with a conflicting options error from the daemon. Why?

A. The --network container:NAME_OR_ID mode shares the target container's network namespace, so flags like --hostname and --add-host are not supported.

B. You can only attach to a stopped container's network stack, not a running one.

C. Sharing a network stack requires both containers to be built from the exact same base image.

D. The -it interactive flags are incompatible with custom network modes.

Question 19. You are migrating a legacy telecom application into Docker containers. The physical network switch and corporate firewall require every application instance to appear as a physical host on the Layer 2 network with its own unique MAC address and routable IP address on the existing LAN. Which Docker network driver should you choose?

A. overlay

B. macvlan

C. ipvlan

D. bridge

Question 20. A developer runs a PostgreSQL container on a laptop connected to a public Wi-Fi network using:

```
docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
```

They only want applications running locally on their own laptop to connect to port 5432, not other devices on the Wi-Fi network. Why is the command above a security risk, and what is the correct fix?

A. -p 5432:5432 binds to all host network interfaces (0.0.0.0) by default; they should use -p 127.0.0.1:5432:5432 to bind strictly to the local loopback interface.

B. -p 5432:5432 disables PostgreSQL password authentication; they should use -P instead.

C. There is no risk because -p binds only to localhost unless --public is specified.

D. They should replace -p 5432:5432 with EXPOSE 5432 in the Dockerfile so only the host browser can connect.

Part 4: Building Images & Running Containers (Questions 21–30)

Question 21. A developer writes a `Dockerfile` for a FastAPI service that includes the instruction `EXPOSE 8080`. After building the image, they start the container using `docker run -d my-fastapi-app`. When they run `curl http://localhost:8080` on the host machine, they receive:

```
curl: (7) Failed to connect to localhost port 8080: Connection refused
```

Why did this happen, and which command resolves it?

- **A.** `EXPOSE 8080` is only metadata documenting the intended port and does not publish the port to the host; they must run `docker run -d -p 8080:8080 my-fastapi-app`.
- **B.** EXPOSE only works for UDP traffic by default; they must change it to `EXPOSE 8080/http`.
- **C.** They must run `docker run -d --expose 8080 my-fastapi-app` to open the host firewall.
- **D.** FastAPI containers cannot use port `8080` unless the `host` storage driver is enabled.

Question 22. Examine the following `Dockerfile` for a Node.js application:

```
FROM node:22-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
```

Every time a developer edits a single HTML string in `src/static/index.html` and runs `docker build .`, the `RUN yarn install --production` step takes 18 seconds to reinstall all packages instead of using the build cache. Why, and how should the `Dockerfile` be optimized?

- **A.** `RUN` instructions are never cached by BuildKit; the developer must commit `node_modules` into Git.
- **B.** Modifying `src/static/index.html` invalidates the `COPY . .` layer, which cascades and invalidates all subsequent layers; they should `COPY package.json yarn.lock ./` and run `RUN yarn install --production` **before** `COPY . .`.
- **C.** Alpine images do not support layer caching; switching to `FROM node:22` fixes the issue.
- **D.** The `WORKDIR /app` instruction clears the build cache on every run.

Question 23. Even after optimizing the `Dockerfile` order in a Node.js project, you notice that running `docker build .` spends several seconds on the following step:

```
=> [internal] load build context
=> => transferring context: 420.50MB
```

Additionally, local files inside your host's `node_modules` folder are overwriting the dependencies installed during the build when `COPY . .` executes. How should you fix this?

- **A.** Remove the dot (`.`) from the end of `docker build .`.
- **B.** Create a `.dockerignore` file in the build context root and add `node_modules` to it.
- **C.** Pass the `--no-cache` flag to `docker build`.
- **D.** Change `COPY . .` to `ADD . .` so Docker automatically filters out dependency folders.

Question 24. In the Docker Build architecture (`Buildx` client + `BuildKit` backend), what is a major efficiency advantage of `BuildKit` compared to the legacy Docker builder regarding build resources?

- **A.** The legacy builder always copies the entire local filesystem context up front, whereas `BuildKit` only requests resources (local files, secrets, SSH sockets) from `Buildx` when the build steps actually need them.
- **B.** `BuildKit` executes all `Dockerfile` instructions directly on the host root filesystem without creating isolated layers.
- **C.** `BuildKit` eliminates the need for a daemon and runs entirely inside the web browser.
- **D.** `BuildKit` automatically compresses all images into a single layer by deleting the base image history.

Question 25. You are writing a multi-stage `Dockerfile` for a Java Spring Boot application to reduce the image size from `880 MB` to `428 MB`:

```
FROM eclipse-temurin:21-jdk-jammy AS builder
WORKDIR /opt/app
COPY . .
RUN ./mvnw clean install
FROM eclipse-temurin:21-jre-jammy AS final
WORKDIR /opt/app
# <INSERT INSTRUCTION HERE>
ENTRYPOINT ["java", "-jar", "/opt/app/app.jar"]
```

Which instruction correctly replaces `# <INSERT INSTRUCTION HERE>` to copy the compiled JAR from the first stage into the final runtime image?

- **A.** `COPY /opt/app/target/app.jar /opt/app/app.jar`
- **B.** `COPY --from=builder /opt/app/target/app.jar /opt/app/app.jar`
- **C.** `RUN cp builder:/opt/app/target/app.jar /opt/app/app.jar`
- **D.** `VOLUME --from=builder /opt/app/target/app.jar`

Question 26. Using the multi-stage `Dockerfile` from Question 25 (which has stages named `builder` and `final`), a CI pipeline needs to build an image that stops at the `builder` stage so it can run Maven debugging tools using the full JDK. Which command accomplishes this?

- **A.** `docker build -t app-debug --stage=1 .`
- **B.** `docker build -t app-debug --target builder .`
- **C.** `docker build -t app-debug --only builder .`
- **D.** `docker run --target builder eclipse-temurin:21-jdk-jammy`

Question 27. You already have one PostgreSQL container running via:

```
docker run -d --name pg-dev -e POSTGRES_PASSWORD=secret -p 5432:5432
postgres
```

You now want to start a second, independent PostgreSQL container (`pg-test`) on the same host machine using the default Postgres configuration inside the container. When you run `docker run -d --name pg-test -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres`, it fails with:

```
Bind for 0.0.0.0:5432 failed: port is already allocated.
```

Which command properly starts `pg-test` without port conflict?

A. `docker run -d --name pg-test -e POSTGRES_PASSWORD=secret -p 5432:5433 postgres`
B. `docker run -d --name pg-test -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres`
C. `docker run -d --name pg-test -e POSTGRES_PASSWORD=secret --expose 5433 postgres`
D. `docker run -d --name pg-test -e POSTGRES_PASSWORD=secret --network host postgres`

Question 28. On a shared staging server, a rogue Python API container occasionally leaks memory and spikes CPU usage, starving other containers. Which `docker run` flags restrict the container to a maximum of `512 MB` of RAM and half of a CPU core, and which CLI command lets you monitor its live resource consumption?

A. Flags: `--ram="512m" --cpu-limit="50%"` | Monitor command: `docker top`
B. Flags: `--memory="512m" --cpus="0.5"` | Monitor command: `docker stats`
C. Flags: `--max-mem=512 --max-cpu=0.5` | Monitor command: `docker inspect`
D. Flags: `-e MEMORY=512m -e CPUS=0.5` | Monitor command: `docker ps -s`

Question 29. You initialize a PostgreSQL 18 container with a named volume:

```
docker run --name=db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/
postgres:18
```

You create a `tasks` table, insert two rows, and then stop and delete the container (`docker stop db && docker rm db`). Next, you start a new container attaching the same volume **without** passing `-e POSTGRES_PASSWORD=secret`:

```
docker run --name=new-db -d -v postgres_data:/var/lib/postgresql postgres:18
```

What happens when `new-db` starts?

A. `new-db` immediately exits with an error because `POSTGRES_PASSWORD` is required on every container startup.
B. `new-db` starts normally and retains the `tasks` table and existing credentials because `POSTGRES_PASS` is only used when bootstrapping a brand-new, empty database directory.
C. `new-db` wipes `postgres_data` and creates an unauthenticated database.
D. Docker blocks the volume mount because the container name changed from `db` to `new-db`.

Question 30. You are deploying a multi-container counter application consisting of an Nginx reverse proxy (`nginx`), two Node.js backend instances (`web1` and `web2`), and a Redis database (`redis`). Why is managing this stack with a `compose.yaml` file (`docker compose up -d --build`) preferred over running four separate `docker run` commands?

A. Because `docker run` does not allow more than two containers to connect to a custom bridge network.
B. Because Docker Compose merges `nginx`, `web1`, `web2`, and `redis` into a single container process so they share a single PID.
C. Because Docker Compose declaratively centralizes service configurations, automatically creates the shared default network and volumes, manages service startup/teardown order, and reconciles configuration changes without manual command-line flags.
D. Because Nginx round-robin load balancing only works when started via Docker Compose.

Answer Key & Detailed Technical Explanations

Quick Reference Table

Q1: B	**Q6:** B	**Q11:** B	**Q16:** B	**Q21:** A	**Q26:** B
Q2: B	**Q7:** B	**Q12:** B	**Q17:** A	**Q22:** B	**Q27:** B
Q3: B	**Q8:** B	**Q13:** B	**Q18:** A	**Q23:** B	**Q28:** B
Q4: B	**Q9:** B	**Q14:** B	**Q19:** B	**Q24:** A	**Q29:** B
Q5: B	**Q10:** B	**Q15:** A	**Q20:** A	**Q25:** B	**Q30:** C

Detailed Explanations

Q1. Correct Answer: B. When `exit` terminates `/bin/bash`, the container stops (`Exited`) but is **not** removed (unless `--rm` was passed). `docker ps` only lists running containers; running `docker ps -a` reveals the stopped container, and `/tmp/debug.log` remains in its writable layer until `docker rm` is executed.

Q2. Correct Answer: B. Unlike Virtual Machines (which boot a separate guest OS kernel per VM), all containers on a host share the host machine's OS kernel and isolate processes via Linux kernel namespaces. Thus, `uname -r` reports the host's kernel version (`6.8.0-45-generic`).

Q3. Correct Answer: B. Docker uses a Client-Server architecture. The `docker` CLI client sends REST API requests over `unix:///var/run/docker.sock` to the Docker daemon (`dockerd`). If `dockerd` is not running (or the user lacks permission to access the socket), the client cannot connect.

Q4. Correct Answer: B. Registries transfer and display the compressed size of image layers to save bandwidth, whereas `docker image ls` displays the total uncompressed size of the extracted layers on the host disk.

Q5. Correct Answer: B. By design, `docker compose down` stops and removes containers and the default network, but preserves named volumes so database data is not accidentally wiped. To remove volumes as well, run `docker compose down --volumes`.

Q6. Correct Answer: B. A full image name follows `[HOST[:PORT]/]PATH[:TAG]`. When no namespace is provided (`payment-service:1.0`), Docker defaults to the `library` namespace (`docker.io/library/payment-service:1.0`), which is reserved for Docker Official Images. It must be tagged as `alexdev/payment-service:1.0`.

Q7. Correct Answer: B. A `Dockerfile` contains instructions to build a container image, whereas a Compose file (`compose.yaml`) declaratively defines running containers (services), networks, and volumes.

Q8. Correct Answer: B. When an **empty volume** is mounted into a non-empty directory inside a container, Docker automatically copies (propagates) the container's existing files into the volume. Conversely, when a host directory is **bind-mounted** onto a non-empty container directory, the container's existing files are obscured (hidden) by the contents of the host directory.

Q9. Correct Answer: B. This is a critical behavioral difference documented in Docker Docs: `-v` automatically creates a missing host directory as an empty folder, whereas `--mount`

`type=bind` produces an explicit error if the `source` path does not exist on the host. Docker recommends `--mount` to avoid silent issues with missing paths.

Q10. Correct Answer: B. Storage drivers like `overlay2` implement Copy-on-Write (CoW). When a file in a lower read-only layer is modified for the first time, the driver searches down the layers and executes a `copy_up` operation that copies the **entire file** to the container's writable layer before modifying it.

Q11. Correct Answer: B. A `tmpfs` mount stores data directly in host memory (RAM) and never writes to disk. While `--tmpfs` does not accept configurable options like size or mode, `--mount type=tmpfs` allows specifying `tmpfs-size` and `tmpfs-mode`.

Q12. Correct Answer: B. Image mounts (`--mount type=image`) are always read-only, require the `containerd` image store to be enabled, only work with the `--mount` flag, and require the source image (`busybox:latest`) to be pulled locally beforehand because Docker does not auto-pull missing images for image mounts.

Q13. Correct Answer: B. Switching to the `containerd` image store changes the storage backend to `containerd` snapshotters, which use a separate directory on disk. Images and containers created under the classic `overlay2` driver remain on disk in the old directory but are not visible while `containerd-snapshotter` is enabled.

Q14. Correct Answer: B. When pulling an image, `containerd` stores both the compressed image layers (as fetched from the registry) and the extracted/unpacked layers on disk, increasing disk usage compared to classic graph drivers.

Q15. Correct Answer: A. A volume cannot be removed while it is attached to any container—even if that container is stopped. Running `docker rm 7a070c9ffeaa` (or `docker rm -f 7a070c9ffeaa`) removes the container so `docker volume rm postgres_data` can succeed.

Q16. Correct Answer: B. Containers without a `--network` flag attach to the default `bridge` network, where containers can only communicate via IP address (no automatic container-name DNS resolution). Creating and attaching both containers to a custom user-defined network enables automatic name resolution via Docker's embedded DNS.

Q17. Correct Answer: A. On custom networks, Docker configures containers to query its embedded DNS server at `127.0.0.11` (even on IPv6-only networks). It resolves container names/aliases within the same network and forwards external queries to the host's configured DNS servers.

Q18. Correct Answer: A. When using `--network container:NAME_OR_ID`, the new container joins the existing container's network stack completely. Options that modify network identity, such as `--hostname` and `--add-host`, are unsupported and cause an error.

Q19. Correct Answer: B. The `macvlan` driver assigns a unique MAC address to each container's virtual network interface, making it appear as a physical device directly on the underlying Layer 2 network. (`ipvlan` shares the host interface's MAC address).

Q20. Correct Answer: A. Publishing a port with `-p 5432:5432` binds to `0.0.0.0` (all host interfaces) by default, exposing the port to external traffic reaching the host. Prefixing `127.0.0.1` (`-p 127.0.0.1:5432:5432`) restricts access strictly to the host's loopback interface.

Q21. Correct Answer: A. The `EXPOSE` instruction in a `Dockerfile` serves as metadata indicating which port the packaged application listens on, but it does not publish the port to

the host machine. You must pass `-p 8080:8080` (or `-P` for an ephemeral host port) in `docker run`.

Q22. Correct Answer: B. Docker invalidates a `COPY` layer whenever any copied file changes, and once one layer is invalidated, all subsequent layers in the `Dockerfile` are also invalidated. Copying `package.json` and `yarn.lock` and running `RUN yarn install --production` **before** `COPY . .` preserves the cached dependency layer when source code changes.

Q23. Correct Answer: B. Adding `node_modules` to a `.dockerignore` file prevents the client from transferring the heavy local `node_modules` directory into the build context and stops `COPY . .` from overwriting container dependencies.

Q24. Correct Answer: A. Unlike the legacy builder (which always copies the entire local filesystem context at the start of a build), `BuildKit` only requests specific resources (files, build secrets, SSH sockets, registry tokens) from the `Buildx` client on demand when needed.

Q25. Correct Answer: B. In multi-stage builds, `COPY --from=<stage-name> <src> <dest>` extracts only the compiled artifact from a previous build stage (`builder`) into the clean runtime stage (`final`), leaving behind the JDK and Maven build tools.

Q26. Correct Answer: B. By default, `docker build` builds up to the final stage in a multi-stage `Dockerfile`. Passing `--target <stage-name>` (`--target builder`) instructs Docker to build and stop at that specific stage.

Q27. Correct Answer: B. In `-p HOST_PORT:CONTAINER_PORT`, Postgres inside the second container still listens on its standard container port 5432, while mapping it to an available host port (5433) via `-p 5433:5432` prevents host port collisions.

Q28. Correct Answer: B. The `--memory="512m"` and `--cpus="0.5"` flags on `docker run` enforce memory and CPU quotas, and `docker stats` streams real-time CPU, memory, network, and disk I/O metrics for running containers.

Q29. Correct Answer: B. As highlighted in the Docker documentation walkthrough, the `POSTGRES_PASSW` environment variable is only used when bootstrapping/initializing a brand-new database. Since `postgres_data` already contains initialized database files from the first container, `new-db` starts normally and retains all existing data and authentication settings.

Q30. Correct Answer: C. Docker Compose allows you to declaratively define all services, networks, volumes, and environment variables in a single `compose.yaml` file, avoiding error-prone manual `docker network create` and multiple `docker run` commands while making scaling and cleanup (`docker compose down`) effortless.